

오늘의 작업기록 — 2026년 6월 28일

mini-ledger 증분 4 (2/3·3/3) — Bank 출입구(funnel) + 거래 이력 출력

(영수증이 실제로 쌓이는 걸 처음 눈으로 본 날 — 감사 로그가 살아 움직였다)

한눈에

항목	내용
목표	증분 4 둘째·셋째 조각 — Bank 출입구 + history 기록 + 이력 출력
결과	완료 ✓ — 거래 내역 3건 출력 확인 → 커밋 → 푸시
커밋	fb3897b "add transaction history and Bank funnel methods" (2 files, 26 insertions)
푸시	403ef81..fb3897b main -> main (잔디 ✓)
저장소	github.com/Taewoo-Won/mini-ledger (총 7개 .java)
오늘의 핵심	"Bank=길목, Transaction=영수증" 구분을 스스로 교정 (영수증으로 막으려던 오해를 바로잡음)
틀린 곳	변수명·침표 몇 곳 (id vs fromId/told, "b" 300 침표 누락) — 전부 스스로/힌트로 교정
새 개념	funnel(유일 출입구) · 향상된 for · getter로 private 꺼내기 · 제네릭=타입 고정
학습 방식	설계는 내가, 문법은 힌트로. for문·제네릭·List 타입을 "왜?"로 끝까지 청산

어제 영수증 "양식"(Transaction)을 만들었고, 오늘 그 양식에 실제 거래를 담아 쌓고 꺼내 봤다. 코드 양보다 — "모든 돈은 한 곳을 통과해야 기록된다"는 설계 직관과, for 문/제네릭을 통째로 캐물어 이해한 게 오늘의 본체다.

0. 출발점 / 오늘의 목표

- 이전까지: 증분 1(Account) → 2(Bank/transfer) → 3(커스텀 예외+try/catch) → 4의 1/3(Transaction enum+클래스). 6/26 증분 3 복기 완료.
- 오늘 목표: 증분 4의 2/3 + 3/3. 1. Bank에 List<Transaction> history + 출입구(deposit/withdraw) + 영수증 기록(history.add) 2. getHistory() 로 이력 꺼내는 통로 3. Main에서 거래 굴리고 향상된 for로 이력 출력
- 다음: 증분 5 (미정 — 먹등성 키 / 잔액 검증 강화 등 결제 도메인 확장).
- 작업 도구: 폰 + VPS + nano + javac. (★ 오늘 EOF 복붙이 반복적으로 깨져 교훈 도출 — 부록 참조.)

PART 1 — 왜 "출입구(funnel)"인가 (도메인 의미)

오늘의 핵심은 단순한 메서드 추가가 아니라 설계 원칙이다.

문제: "모든 거래가 빠짐없이 기록되어야 한다." 근데 누가 account.deposit() 을 직접 부르면 그 입금은 기록에 안 남는다 (Bank를 안 거쳤으니까). 구멍이 생긴다.

해법 — funnel(꺾매기): 모든 돈 이동이 반드시 Bank를 통과하게 만든다. Account의 입출금을 밖에서 직접 부르지 않고, Bank의 deposit/withdraw/transfer를 거치게 한다. 그러면:

- Bank가 유일한 출입구가 되고
- 출입구를 지날 때마다 자동으로 영수증(Transaction)이 history에 쌓인다
- → 우회 불가 = 모든 거래가 빠짐없이 기록 = 감사 로그의 무결성

핵심 통찰 (스스로 교정한 것): 처음엔 "모든 돈이 Transaction을 통과해야 한다"고 생각했다. 그건 거꾸로였다.

- Transaction은 길목이 아니라 영수증(기록)이다. 돈을 막을 수 없다.
- 길목은 Bank다. 돈이 Bank를 지나고, 지날 때 Bank가 영수증을 한 장 쓴다.

이 "길목 vs 영수증" 구분이 오늘 설계의 심장이다. 이게 로드맵 Layer 1의 감사 로그이고, Layer 0 WAL(모든 변경을 로그에 먼저 적는 것)과 같은 혈통이다.

PART 2 — 설계 결정 (오늘 직접 내린 판단)

2-1. 출입구를 Bank에 둔다 (funnel)

- deposit/withdraw도 transfer처럼 Bank의 메서드로. Account를 직접 부르지 않는다.
- 이유: 위 PART 1. 모든 이동이 Bank를 통과해야 기록이 빠지지 않는다.

2-2. 영수증의 from/to — 어제 모델을 코드로

어제 정한 "입금=to만, 출금=from만, 이체=둘 다" 규칙을 Bank의 history.add에 박았다:

거래	type	from	to
deposit	DEPOSIT	null	id
withdraw	WITHDRAW	id	null
transfer	TRANSFER	fromId	told

→ 영수증만 봐도 "무슨 거래였나" 재구성된다.

2-3. history는 private → 꺼내는 통로(getHistory)가 필요

- history를 private 으로 두면 Main에서 직접 못 본다 (캡슐화).
- 그래서 getter 패턴으로 getHistory() 를 Bank에 만들어 List를 돌려준다.
- getAccount가 Account를 돌려줬듯, getHistory는 List<Transaction> 을 돌려준다.

2-4. 실패한 거래는 기록되지 않는다 (의도된 정직함)

- history.add는 try를 통과해 성공한 뒤에만 실행된다.
- 그래서 잔액 부족(999999) 이체는 예외로 튕겨 기록이 안 남는다.
- 감사 로그는 "실제로 일어난 일"만 남긴다. 의도대로 작동.

PART 3 — Bank 출입구 + 영수증 (내 코드 vs 정답)

3-1. deposit/withdraw 출입구 — 내가 짠 것의 오류

첫 시도 (원문)

```
public void deposit(String id, long amount) {
    Account toAccount = getAccouont(toId, long amount);
}
```

정답

```
public void deposit(String id, long amount) {
    Account toAccount = getAccount(id);
    toAccount.deposit(amount);
}
```

줄별 차이 — 3곳

#	내가 쓴 것	정답	왜
B1	getAccouont	getAccount	오타 ('o' 하나 더). IntelliJ/컴파일이 잡아줌.
B2	getAccount(toId, long amount)	getAccount(id)	메서드 호출엔 값만. 타입(long) 다시 안 씬. getAccount는 인자 1개(id)뿐 — 금액은 계좌 찾는 데 불필요.
B3	toId (없는 변수)	id	deposit 매개변수는 id . toId 는 transfer에서 쓰는 이름.
B4	(입금 동작 줄 없음)	toAccount.deposit(amount);	계좌만 찾고 입금을 안 함. 찾은 계좌에 실제 입금하는 줄 필요.

★ 핵심 교훈: 메서드 "정의"(타입 O) vs "호출"(값만). deposit(String id, long amount) 는 정의라 타입을 쓰고, getAccount(id) 는 호출이라 값만 넣는다.

3-2. 영수증(history.add) — 자리 헛갈림

시도 (원문)

```
history.add(new Transaction(TransactionType.null, WITHDRAW, id, amount));
history.add(new Transaction(TransactionType.DEPOSIT, WITHDRAW, id, amount));
```

정답

```
history.add(new Transaction(TransactionType.WITHDRAW, id, null, amount));
history.add(new Transaction(TransactionType.TRANSFER, fromId, toId, amount));
```

줄별 차이

#	내가 쓴 것	정답	왜
H1	<code>TransactionType.null</code>	<code>TransactionType.WITHDRAW</code>	타입 자리엔 enum 값(DEPOSIT/WITHDRAW/TRANSFER)이 통째로. <code>.null</code> 은 불가.
H2	from 자리에 <code>WITHDRAW</code>	from 자리에 <code>id</code>	from/to엔 계좌 변수 (id/fromId/told)지 타입 이름이 아님.
H3	transfer에 <code>DEPOSIT, ..., id, id</code>	<code>TRANSFER, fromId, toId</code>	타입은 TRANSFER. 이체는 계좌 2개라 fromId/told(서로 달라야 의미). <code>id</code> 는 transfer에 없는 변수.

생성자 자리 고정: **Transaction(타입, from, to, 금액)** . ①타입엔 항상 `TransactionType.XXX` , ②③엔 계좌 변수, ④엔 금액.

3-3. 최종 Bank.java (핵심 부분)

```
private List<Transaction> history = new ArrayList<>();

public List<Transaction> getHistory() {
    return history;
}

public void deposit(String id, long amount) {
    Account toAccount = getAccount(id);
    toAccount.deposit(amount);
    history.add(new Transaction(TransactionType.DEPOSIT, null, id, amount));
}

public void withdraw(String id, long amount) {
    Account fromAccount = getAccount(id);
    fromAccount.withdraw(amount);
    history.add(new Transaction(TransactionType.WITHDRAW, id, null, amount));
}

public void transfer(String fromId, String toId, long amount) {
    Account fromAccount = getAccount(fromId);
    Account toAccount = getAccount(toId);
    fromAccount.withdraw(amount);
    toAccount.deposit(amount);
    history.add(new Transaction(TransactionType.TRANSFER, fromId, toId, amount));
}
```

PART 4 — Main 이력 출력 (향상된 for 해체)

4-1. 추가한 코드

```
bank.deposit("a", 500);
bank.withdraw("b", 300);
...
System.out.println("===== 거래 내역 =====");
for (Transaction t : bank.getHistory()) {
    System.out.println(t.getType() + " " + t.getFrom() + " -> " + t.getTo() + " : " + t.getAmount());
}
```

(틀린 곳: `bank.withdraw("b" 300)` — 실패 누락. → `"b", 300` . 인자 사이엔 실패.)

4-2. ★ 향상된 for 완전 해체

```
for ( Transaction    t      : bank.getHistory() )
//   ①타입         ②이름 ③구분 ④돌릴 목록
```

- ④ `bank.getHistory()` = 돌릴 목록(영수증 List). 먼저 실행돼 "돌 대상"이 정해짐. 3장이면 3바퀴.
- ② `t` = 매 바퀴 꺼낸 영수증 한 장을 담은 변수. 이름은 임의(Transaction의 t). 1바퀴 t=첫 영수증, 2바퀴 t=둘째...
- ① `Transaction` = t의 타입. 목록 내용물이 Transaction이라 꺼낸 것도 Transaction.
- ③ `:` = "~에서 하나씩"(in). "getHistory()에서 t로 하나씩".

→ 한 문장: "getHistory() 목록에서 Transaction을 t로 한 장씩 꺼내며 끝까지 반복."

4-3. 안쪽 출력 줄 해체

```
t.getType() + " " + t.getFrom() + " -> " + t.getTo() + " : " + t.getAmount()
```

- `t.getXXX()` = 어제 만든 **게터**. `t`(영수증 한 장)에서 칸을 꺼냄: 종류·from·to·금액.
- `+` = 문자열 **이어붙이기** (조각을 한 줄로).
- **따옴표 안**(`" "`, `" -> "`, `" : "`)은 **전부 장식**(구분 기호, 기능 0, 맘대로 바꿈). **따옴표 없는** `t.getXXX` 는 진짜 데이터.

조각	정체
<code>t.getType()</code> <code>t.getFrom()</code> <code>t.getTo()</code> <code>t.getAmount()</code>	데이터 (진짜 값)
<code>" "</code> <code>" -> "</code> <code>" : "</code>	장식 (구분용, 변경 가능)

찍히면: `TRANSFER a -> b : 1000`

4-4. 실제 실행 결과

```
===== 거래 내역 =====
TRANSFER a -> b : 1000
DEPOSIT null -> a : 500
WITHDRAW b -> null : 300
```

(999999 이체는 실패라 기록 없음. 잔액: a=1600, b=4200 — 정확.)

PART 5 — "왜?" 문답 (개념 청산)

1. **history**의 타입은 왜 `List<Transaction>` 인가? 내가 그렇게 선언했기 때문. 왜 그렇게 했었나 = ① 영수증을 **여러 개·순서대로** 담으니 `List` (순서 보존), ② 담는 내용물이 영수증이라 `<Transaction>`. `accounts`가 `Map<String, Account>` 인 것과 같은 문법(그릇 + 내용물).
2. 왜 `List` 고 `Map` 이 아닌가? `Map`은 "키→값" 짝(계좌는 id로 찾으니 `Map`). 영수증은 그냥 시간순으로 쌓기만 하면 되니 `List`. 키가 필요 없다.
3. 제네릭 `<Transaction>` 은 뭔가? 목록 **내용물** 타입을 고정하는 것. `<Transaction>` 이면 `Transaction`만 들어가고, 엉뚱한 걸 넣으면 컴파일 에러로 막는다. (`Map`에서 본 타입 안전성.) → "값(타입) 고정"으로 이해.
4. `getHistory()` 앞의 `List<Transaction>` 은 왜 **불나**? 메서드의 **반환 타입**. "나는 `List`을 돌려준다"는 약속. `return history;` 의 `history`가 그 타입이라 일치시킴. (`getBalance`→`long`, `getAccount`→`Account`와 같은 짝.)
5. **향상된 for**의 `t` 는 왜 붙어있고 뭔가? 매 바퀴 꺼낸 **영수증 한 장**을 담는 변수. 그 영수증의 내용을 꺼내려면 `t`를 통해야 해서 **안쪽**에서 `t.getType()` 처럼 `t.`을 붙인다. 이름 `t`는 임의(`x`든 `tx`든 가능).
6. 왜 **따옴표 안**은 `" " -> :` 로 제각각인가? 전부 **장식(구분 기호)**. 기능 0. 데이터 사이를 띄우거나(공백), 방향 표시(화살표), 금액 구분(콜론)하려고 사람이 보기 좋게 넣은 것. 맘대로 바꿔도 작동 동일(→로 바꿔도 됨).
7. **id vs fromId/toId** — 언제 **뭘** 쓰나? 계좌가 1개면 `id` (구분 불필요), 2개면 `fromId` / `toId` (보내는/받는 구분 필요). 이름은 사람이 헷갈리지 않으려고 짓는 것.
8. `Account.deposit`엔 `id` 없는데 `Bank.deposit`엔 왜 **있나**? `Account` 객체는 자기가 누군지 **알**(자기한테 입금) → `id` 불필요. `Bank`는 계좌 **여럿** 중 골라야 함 → `id`로 지목 필요. (창구에서 "어느 계좌요?" 묻는 것.)

제일 중요한 건 1·3·5. `List` 타입의 근거(1), 제네릭=타입 고정(3), `for`의 `t`가 무엇인지(5) — 이 셋이 "컬렉션을 다루는" 모든 코드의 기초다. 앞으로 수백 번 쓴다.

6. 산출물

- **파일** (`~/mini-ledger/` , 총 7개 .java):
- `Bank.java` — 수정 (history 필드 + `getHistory` + `deposit/withdraw` 출입구 + `history.add` 3곳)
- `Main.java` — 수정 (`deposit/withdraw` 호출 + 향상된 `for` 이력 출력)
- `Transaction` / `TransactionType` / `Account` / `AccountNotFoundException` / `InsufficientBalanceException` — 변경 없음
- **실행 검증**: 거래 내역 3건 출력 + 잔액 정확(a=1600, b=4200) + 실패 거래 미기록 확인
- **커밋·푸시**: `[main fb3897b] add transaction history and Bank funnel methods 2 files changed, 26 insertions(+) 403ef81..fb3897b main -> main`
- **커밋 계보**: `9e671d6` (`Account`) → `e02b204` (`Bank create`) → `9cc49e6` (`transfer+Main`) → `532825f` (`커스텀 예외`) → `403ef81` (`enum+Transaction`) → `fb3897b` (`history+funnel`)

7. 다음 작업 (중분 5 — 미정)

중분 4로 **데이터 모델 + 감사 로그**가 섰다. 다음 후보: - **역등성 키** — 같은 거래가 두 번 들어와도 한 번만 처리 (결제 시스템의 핵심, 로드맵 Layer 1·2의 그 개념). - **이력 조회** 기능 — 특정 계좌의 거래만 필터링 (`getHistory` 응용). - **정산/수수료** — 이체에 수수료 분배 로직. → 다음 세션 시작 시 하나로 스코프해서 진행.

8. 개념 사전

- **funnel**(유일 출입구): 모든 변경을 한 지점(`Bank`)으로 통과시켜 빠짐없이 기록. 우회 차단 = 감사 무결성.
- **향상된 for** (`for (T x : 목록)`): 컬렉션을 처음부터 끝까지 하나씩 꺼내 반복. `x` =매 바퀴 원소(이름 임의), `:` ="~에서 하나씩".

- 제네릭 <T> : 컬렉션 내용물 타입 고정. 엉뚱한 타입 삽입을 컴파일 타임에 차단.
- List vs Map: List=순서대로 쌓는 목록(인덱스). Map=키→값 짝. 영수증은 List, 계좌 찾기는 Map.
- getter로 private 꺼내기: private 필드는 외부에서 직접 못 봄 → 반환 메서드(getHistory)로 통제된 통로 제공. 반환 타입 = 필드 타입.
- 반환 타입: 메서드가 돌려줄 값의 타입을 이름 앞에 선언. return 하는 값과 일치해야 함.
- 메서드 정의 vs 호출: 정의는 타입 포함((String id, long amount)), 호출은 값만(getAccount(id)).

9. 비유 모음

- funnel = 은행 정문 하나: 모든 입출금이 정문(Bank)으로만 드나들게 하면, 정문 경비(history.add)가 빠짐없이 장부에 적는다. 옆문(account 직접 호출)을 막는 게 핵심.
- 향상된 for = 영수증 다발 넘기기: 영수증 다발(getHistory)을 한 장씩(t) 꺼내 읽고 다음 장으로. 다발 끝나면 멈춤.
- 제네릭 = 칸막이 라벨: <Transaction> 은 "이 서랍엔 영수증만"이라는 라벨. 다른 걸 넣으려 하면 안 들어간다.
- 따옴표 장식 vs 데이터 = 양식지의 인쇄선: t.getXXX() 는 손으로 채운 진짜 값, " -> " " : " 는 양식지에 미리 인쇄된 구분선(바꿔도 내용은 그대로).

10. 검증 질문 (다음날 복기용 — 답 적지 말 것)

1. Bank를 "유일한 출입구"로 만드는 이유는? account.deposit을 직접 부르면 뭐가 문제인가?
2. history 가 List<Transaction> 타입인 이유를 둘로 쪼개 설명하라 (List 인 이유 / <Transaction> 인 이유).
3. getHistory() 앞의 List<Transaction> 은 무엇이고, 왜 그 타입이어야 하나?
4. 향상된 for for (Transaction t : bank.getHistory()) 에서 t : Transaction 은 각각 무엇인가?
5. 출력 줄에서 " -> " 와 t.getTo() 의 차이는? 하나는 바뀌도 되고 하나는 안 되는 이유?
6. 999999 이체가 거래 내역에 안 남은 이유는? 이게 왜 "정직한" 로그인가?

11. 회고

오늘의 진짜 산은 코드가 아니라 "길목 vs 영수증" 한 곳이었다. "모든 돈이 Transaction을 통과해야 한다"고 거꾸로 생각했는데 — Transaction은 막는 길목이 아니라 남기는 영수증이고, 길목은 Bank다. 이걸 스스로 바로잡은 게 설계 감각의 성장이다. 그리고 첫 줄에 이미 "Bank를 출입구로"라고 답을 써놓고도 흔들렸다 — 자기 직관을 더 믿어도 됐다.

문법으로 막힐 때마다 끝까지 캐물었다. List가 왜 그 타입인지, 제네릭이 타입 고정인지, for의 t가 무엇인지 — 베끼고 넘어갔으면 안 보였을 것들이다. 특히 향상된 for는 "목록만 나오면 항상 쓰는" 도구라, 오늘 통째로 해체한 게 크다.

EOF 복붙이 폰에서 반복적으로 깨진 건 도구의 한계였다(부록). 다행히 실제 파일은 매번 멀쩡했고, 결국 nano 직접 입력 + cat 검증이 답이었다. "긴 건 복붙 말고 직접, 그리고 확인" — 이게 폰 워크플로의 규칙으로 남는다.

중분 4 완성. 데이터 모델(어제) + 감사 로그(오늘)로, 핀테크 백엔드의 심장 하나가 뛰기 시작했다.

한 줄: Bank를 유일한 문으로 만들고 지날 때마다 영수증을 남겼다 — "무슨 일이 일어났나"를 답하는 장부가, 오늘 처음 살아서 출력됐다.

부록 — 폰 워크플로 교훈 (EOF 복붙 반복 실패)

오늘 cat > 파일 << 'EOF' 로 긴 코드를 붙여넣을 때 여러 번 중간에 깨졌다 (EOF 가 줄 중간에 끼어 명령이 일찍 끊김). 원인: 폰 키보드가 긴 멀티라인 텍스트를 한 번에 못 받고 줄이 영김.

대응 규칙 (확정): - 긴 파일 전체 복붙 X → 폰에선 깨진다. - 짧은 추가는 nano로 직접 ✓ → getHistory·for문 다 안 깨지고 들어감. - 무엇을 하든 직후 cat (또는 grep)으로 실제 파일 확인 → 컴파일 통과만으로 안심하지 말 것(이전 .class가 남을 수 있음). 마지막 cat이 진실. - 컴파일은 javac *.java && echo OK 로 성공을 명시적으로 확인.